Issue 1 — Priority score weights don't add up to 1.0

The current weights: 0.35 + 0.20 + 0.15 + 0.15 + 0.10 + 0.05 = **1.00** ✓ That's fine. But there's a hidden problem: `priority_score` is listed as both an *output* of Module 3 and an *input feature* to the XGBoost model in Module 4. This is a data leakage risk. If you train XGBoost with `priority_score` as a feature, the model is partially predicting the label using a manually constructed score that already encodes severity and junction weight. Either remove `priority_score` from Module 4 features and keep the raw components (severity, junction_weight, etc.) as separate features, or use `priority_score` alone and drop the individual components. Using both is redundant and leaky.

Issue 2 — DBSCAN epsilon is undefined

The roadmap correctly switches to haversine + ball_tree, but never specifies how epsilon will be chosen. Your K-distance plot is done but the output isn't shown in the notebook — the cell just shows the plot was generated. You need to read the elbow off that plot and convert it to radians: `epsilon_radians = epsilon_km / 6371`. For Bangalore's density, you'll likely find the elbow between 0.05–0.15 km. Document the chosen value and justify it from the K-distance plot. Judges will ask "how did you pick epsilon?" and "we looked at the K-distance plot and chose the elbow" is the correct answer.

Issue 3 — validation_status NaN scope is larger than stated

The roadmap says NaN ≈ not-yet-reviewed. That's plausible, but your EDA shows something worth checking: the crosstab shows NaN records exist across all months (Nov, Dec, Jan, Feb, Mar). If NaN were purely a backlog, you'd expect them to cluster in recent months. Since they're spread across all 5 months, it could mean a structural issue — perhaps a system that only captures validation for certain enforcement centers. Before committing to `approved + NaN`, add one line to your data filtering step:

python

```
df.groupby("center_code")["validation_status"].apply(
    lambda x: x.isna().mean()
).sort_values(ascending=False)
```

If certain center codes have 100% NaN, they likely never push validation data — including them is fine. If NaN is random across all centers, that's a different concern. This one check makes your filtering decision fully defensible.

Issue 4 — Multi-violation score needs a parsing step that isn't specified

The `violation_type` column is stored as a string like `["WRONG PARKING","PARKING IN A MAIN ROAD"]`, not an actual list. Your EDA uses `ast.literal_eval` to parse it. The roadmap mentions multi-violation bonus but never addresses this parsing requirement in the feature engineering step. Before you can compute `num_violations` or `max_severity`, you need:

python

```python
import ast
df["violations_list"] = df["violation_type"].apply(ast.literal_eval)
df["num_violations"] = df["violations_list"].apply(len)
df["max_severity"] = df["violations_list"].apply(
    lambda v: max(severity_weight.get(x, 0) for x in v)
)
```

This should be explicitly listed in the "Immediate Next Step" sequence between step 1 (filtering) and step 3 (DBSCAN), because severity scoring feeds the risk engine and DBSCAN just needs coordinates.

Issue 5 — Vehicle type mapping has mismatches with actual data

The roadmap's `vehicle_weight` dict uses keys like `"MOTORCYCLE"` and `"AUTO"`, but your EDA shows the actual column values are `"MOTOR CYCLE"` (with a space) and `"PASSENGER AUTO"`. The mapping will silently fail and assign no weight to ~78,000 records (MOTOR CYCLE: 40,811 + PASSENGER AUTO: 37,813). Align the keys exactly to the EDA output. The full correct mapping based on your actual data:

python

```python
vehicle_weight = {
    "SCOOTER": 1.0,
    "MOPED": 1.0,
    "MOTOR CYCLE": 1.0,      # not "MOTORCYCLE"
    "PASSENGER AUTO": 1.8,   # not "AUTO"
    "GOODS AUTO": 1.8,
    "CAR": 1.5,
    "JEEP": 1.5,
    "VAN": 2.0,
    "MAXI-CAB": 2.0,
    "TEMPO": 2.5,
    "LGV": 2.5,
    "MINI LORRY": 3.0,
    "PRIVATE BUS": 3.5,
    "BUS (BMTC/KSRTC)": 3.5,
    "TOURIST BUS": 3.5,
    "SCHOOL VEHICLE": 3.5,
    "FACTORY BUS": 3.5,
    "HGV": 4.0,
    "LORRY/GOODS VEHICLE": 4.0,  # not "LORRY"
    "TANKER": 4.0,
    "TRACTOR": 3.0,
    "OTHERS": 1.5
}
```

This is the single highest-risk silent bug in the entire roadmap — it would produce wrong scores for 26% of your records.

Three things that are correct and don't need changing:

The haversine DBSCAN switch is right. The decision to classify (Low/Medium/High/Critical) rather than regress is right. The impact simulator scope — one slider, risk score reduction only, no traffic flow claims — is exactly the right level of ambition for a hackathon.